



# The Quantum Millionaire Mind

*Secrets of Blockchain Wealth  
from the Q-NarwhalKnight Founder*

Inspired by T. Harv Eker's *Secrets of the Millionaire Mind*

Applied to building a quantum-resistant blockchain from zero to mainnet

**Abstract.** Every person carries a financial blueprint—a set of beliefs and habits etched in the subconscious that determines their financial destiny. T. Harv Eker taught that to change your financial life, you must first rewrite your money blueprint. This paper tells the story of a founder who did exactly that—not in a journal, but in Rust. Eighty-plus crates. A 21-million-coin emission schedule spanning 256 years. Post-quantum cryptography for threats decades away. And a transparent 1.9% founder fee encoded immutably in consensus, so that every gigahash of mining power becomes passive income—not a salary, but a financial blueprint made permanent in code.

Where Eker rewired his thermostat with affirmations, this founder rewired his with u128 arithmetic and 4,000 tests.

**Q-NarwhalKnight Whitepaper**

February 2026

Version 1.0

Q-NarwhalKnight Development Team  
<https://quillon.xyz>

## Contents

|    |                                                          |    |
|----|----------------------------------------------------------|----|
| 1  | Your Code Is Your Financial Blueprint                    | 2  |
| 2  | Rich People Think Big, Poor People Think Small           | 3  |
| 3  | Rich People Are Committed, Poor People Want to Be Rich   | 4  |
| 4  | The Wealth File: 1.9 Percent of Everything               | 6  |
| 5  | Rich People Manage Their Money, Poor People Mismanage It | 7  |
| 6  | Rich People Have Their Money Work Hard for Them          | 9  |
| 7  | Rich People Associate with Positive, Successful People   | 10 |
| 8  | Rich People Are Willing to Promote Themselves            | 11 |
| 9  | Rich People Act in Spite of Fear                         | 13 |
| 10 | Rich People Constantly Learn and Grow                    | 14 |
| 11 | After the Last Coin: The Post-Emission Economy           | 16 |
| 12 | Governance: Who Controls the Thermostat?                 | 17 |

## Part I: The Blueprint

### 1 Your Code Is Your Financial Blueprint

“Your income can grow only to the extent that you do.”

—T. Harv Eker, *Secrets of the Millionaire Mind*

#### ★ Wealth File #1

*“Your money blueprint determines your financial life. If your blueprint is set for earning \$50,000 a year, that’s exactly what you’ll make—not a penny more. To change your financial life, you must first change your financial blueprint.”*

Most people’s financial blueprints live in their subconscious—inherited from parents, shaped by childhood, reinforced by society. They are invisible and, precisely because of that, almost impossible to change.

But what if you could write your financial blueprint in a language so precise that it compiles to machine code? What if your relationship with money were not a vague feeling but a set of constants, checked by a compiler, verified by 4,000 tests, and enforced by cryptographic consensus across every node on the planet?

That is exactly what the founder of Q-NarwhalKnight did.

The blueprint began with three lines of Rust:

Listing 1: The Financial Blueprint in Code (`emission_controller.rs`)

```
1 /// Maximum supply: 21,000,000 QUG with 24 decimal places
2 pub const QUG_MAX_SUPPLY: u128 =
3     21_000_000_000_000_000_000_000_000_000;
4
5 /// Base annual emission for Era 0: 2,625,000 QUG
6 pub const BASE_ANNUAL_EMISSION: u128 =
7     2_625_000_000_000_000_000_000_000_000;
8
9 /// Seconds per halving era: 4 years (Julian)
10 pub const SECONDS_PER_HALVING: u64 = 126_230_400;
```

Twenty-one million coins. Four-year halving cycles. A 256-year emission schedule computed with `u128` fixed-point arithmetic at  $10^{24}$  decimal precision. No floating-point rounding errors. No ambiguity. No “I think I deserve more.” The entire monetary universe of QUG is defined by three constants and the math that flows from them.

Eker says your blueprint is your thermostat—set it to 72 degrees and the system will always return to 72, no matter how hot or cold it gets outside. In Q-NarwhalKnight, the thermostat is the `EmissionController`: a feedback loop that measures real block rates, computes the ideal reward, applies an error-correction factor, and ensures the network converges on exactly 2,625,000 QUG per year in Era 0—no more, no less.

Most thermostats are invisible. This one is open-source.

### The Principle

Most people inherit a financial blueprint they never examine. The founder's first act was to write one from scratch—in a language that refuses to compile ambiguity.

## 2 Rich People Think Big, Poor People Think Small

“If you shoot for the stars, you'll at least hit the moon.”

—T. Harv Eker

### ★ Wealth File #2

*“Big thinking and big actions lead to having both money and meaning. The size of the problem is never the issue—what matters is the size of you.”*

When the founder started writing Q-NarwhalKnight, the sensible advice was: “Just fork Bitcoin.” Or: “Use Substrate.” Or: “Nobody needs another blockchain—and certainly not one with quantum cryptography for threats that won't exist for twenty years.”

The founder thought big anyway. Sensible advice, after all, is the financial blueprint of the middle class:

- **80+ Rust crates** across consensus, networking, storage, mining, privacy, DEX, smart contracts, AI inference, TUI, GUI, Tor integration, and bounty systems.
- **Post-quantum cryptography**—Dilithium5, Kyber1024, SPHINCS+—for threats decades away, because “the best time to prepare for a quantum computer is before one exists.”
- **A 256-year emission schedule** spanning 64 halving eras, with mathematical precision that accounts for leap years using the Julian-year convention ( $365.25 \times 86,400 = 31,557,600$  seconds).
- **Tor privacy circuits**—four dedicated circuits per validator, rotated every epoch, with a Dandelion++ gossip protocol for traffic-analysis resistance.
- **An embedded DEX** with AMM liquidity pools, 24-decimal precision arithmetic, and overflow-safe u128 math.
- **Smart contracts** via a WASM virtual machine with gas metering and a collateral-vault system.

Every one of these was built from scratch—not forked, not copied, not “inspired by.” Eighty crates, each with its own `Cargo.toml`, its own test suite, its own reason for existing.

Eker would recognize this instantly: it is the difference between a “small-thinking” fork that copies someone else's blueprint and a “big-thinking” architecture that dares to define its own.

The workspace `Cargo.toml` tells the story:

Listing 2: The Scale of Ambition (`Cargo.toml`, abbreviated)

```
1 [workspace]
2 members = [
3     "crates/q-api-server",      # REST API + block producer
4     "crates/q-storage",        # RocksDB engine + emission
5     "crates/q-network",        # libp2p gossipsub + DHT
6     "crates/q-types",          # Consensus types + crypto
```

```

7   "crates/q-mining",           # Mining engine + dev fee
8   "crates/q-dex",             # AMM DEX with LP pools
9   "crates/q-tor-client",      # Embedded Tor (arti)
10  "crates/q-tor-circuit",     # Dedicated circuit mgmt
11  "crates/q-vm",              # WASM smart contracts
12  "crates/q-stablecoin",      # Algorithmic stablecoin
13  "crates/q-dandelion",       # Dandelion++ privacy
14  "crates/q-bounty-protocol", # Bug bounty system
15  # ... 70+ more crates
16 ]

```

### The Principle

Small thinkers fork. Big thinkers build from first principles. The size of your codebase reflects the size of your vision.

## 3 Rich People Are Committed, Poor People Want to Be Rich

“The number one reason most people don’t get what they want is that they don’t know what they want. Rich people are totally clear that they want wealth.”

—T. Harv Eker

### ★ Wealth File #3

*“There is a difference between being interested in something and being committed to it. When you’re interested, you do what’s convenient. When you’re committed, you accept no excuses—only results.”*

Commitment is not glamorous. Commitment is a 3 AM debug session when the emission controller has a 34× overshoot because someone added three extra zeros to `BASE_ANNUAL_EMISSION`:

### The 34× Emission Overshoot (Feb 9, 2026)

The emission controller was minting 34 times the intended supply. The constant `BASE_ANNUAL_EMISSION` had  $10^{27}$  instead of  $10^{24}$ —three zeros too many in a 31-digit number. Meanwhile, `add_block()` was defined but never called, so the block-rate measurement was wrong too.

The fix required:

1. Correcting the constant (count every zero in a 24-decimal number)
2. Wiring `track_block_for_emission()` into `balance_consensus.rs` for all blocks—local and P2P
3. Verifying the fix didn’t break 4,000+ existing tests

Commitment is the 26.9 GB memory leak at 2 AM:

### The RocksDB Memory Leak (Feb 19, 2026)

Beta server grew to 26.9 GB RSS and got OOM-killed. RocksDB was auto-configuring its block cache to one-third of total RAM—16 GB on the production server. The fix: `ROCKSDB_BLOCK_CACHE_MB=4096` in the systemd service file. Four characters of configuration. Three hours of diagnosis. That ratio—hours of detective work for a one-line fix—is the hidden cost of every line of infrastructure code ever written.

Commitment is the height regression that nearly wiped the chain:

### The Height Regression (Feb 19, 2026)

Node restarted at height 5,431 and came back at 2,502. The fast-recovery threshold was `> 10_000`—at height 5,431, the node fell through to a genesis scan that returned 2,502. Additionally, `save_safe_floor()` used a non-synced `put()` instead of `put_sync()` with `fsync`. The write looked successful but never reached disk.

`cargo clean && cargo build -release`—while the world sleeps.

Eker's distinction is precise: wanting to be rich is a fantasy. Commitment means you fix the bug at 3 AM because 27 miners depend on your node. You do not complain. You do not take a break. You `cargo clean -package q-storage` and rebuild, because incremental compilation cached the old constant and you cannot trust it.

A wish compiles to nothing. Commitment compiles to a release binary.

### The Principle

Wanting to build a blockchain is a wish. Commitment is counting zeros in a 31-digit constant at 3 AM and not stopping until the tests pass.

## Part II: The 1.9% Principle

### 4 The Wealth File: 1.9 Percent of Everything

“Rich people focus on their net worth.  
Poor people focus on their working income.”

—T. Harv Eker

#### ★ Wealth File #4

*“The true measure of wealth is net worth, not working income. Working income is important, but it is only one of four factors in your net worth—and not the most important one. Net worth includes income, savings, investments, and simplification.”*

The founder could have paid themselves a salary. A monthly wire transfer. An hourly rate. That is what employees do—trade time for money, the financial blueprint of the working class. Instead, the founder wrote seven lines of Rust:

Listing 3: The 1.9% Fee in Consensus (`block_producer.rs`)

```
1 // v7.1.5: Configurable dev fee (basis points)
2 let dev_fee_bps_val = self.dev_fee_bps
3   .load(std::sync::atomic::Ordering::Relaxed) as u128;
4 const BPS_DIVISOR: u128 = 10_000;
5 const FOUNDER_WALLET_HEX: &str =
6   "efca1e8c1f46e91013b4073898c771bb3d566453537ccf87\
7   e834505925e50723";
8
9 // Integer-only fee calculation for cross-platform determinism
10 let dev_fee_amount = total_reward
11   .saturating_mul(dev_fee_bps_val) / BPS_DIVISOR;
12 let miner_reward = total_reward
13   .saturating_sub(dev_fee_amount);
```

Not a salary. A *percentage*. 1.9% of every block reward, from every miner, on every node, for as long as the chain lives—or until the last of the 21,000,000 QUG is minted, approximately 256 years from now.

Let the math speak:

Table 1: The 1.9% Fee Across the Emission Schedule

| Era                       | Annual Emission (QUG) | 1.9% (QUG/yr)  | Miners (QUG/yr)   |
|---------------------------|-----------------------|----------------|-------------------|
| 0 (Years 1–4)             | 2,625,000             | 49,875         | 2,575,125         |
| 1 (Years 5–8)             | 1,312,500             | 24,937         | 1,287,563         |
| 2 (Years 9–12)            | 656,250               | 12,469         | 643,781           |
| 3 (Years 13–16)           | 328,125               | 6,234          | 321,891           |
| <b>Lifetime (64 eras)</b> | <b>21,000,000</b>     | <b>399,000</b> | <b>20,601,000</b> |

399,000 QUG over 256 years. That is 1.9% of the entire monetary supply, distributed block by block, reward by reward. Not taken in a pre-mine. Not allocated in a whitepaper and forgotten. *Earned*, continuously, from the productive output of the network.

Compare this with the industry norm: Ethereum’s foundation held 12% of the initial supply. Solana insiders retained 48%. Ripple pre-mined 100% and distributed at its discretion. The Q-NarwhalKnight founder pre-mined nothing—every coin is emitted through proof-of-work, and the 1.9% is deducted transparently, on-chain, from each block reward.

The wallet address `efca1e8c...` is hardcoded into consensus. Every node validates it. Every miner sends to it. It is not a secret—it is the most transparent founder fee in blockchain history.

*Every block, every miner, every node*

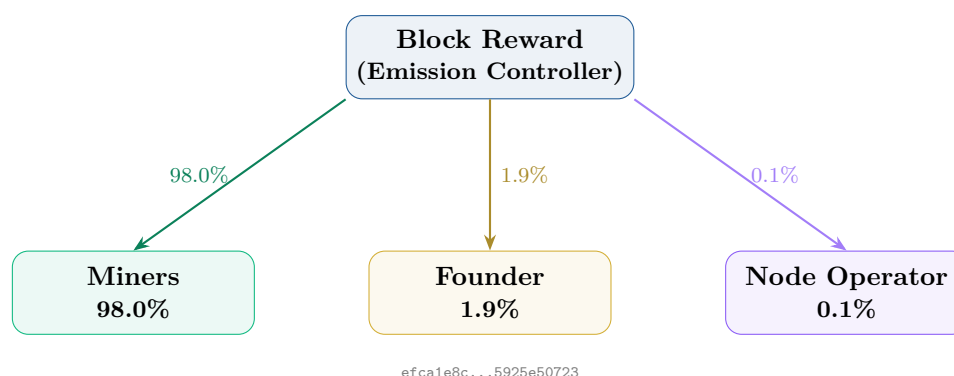


Figure 1: Fee flow: block reward distribution across the network.

### The Principle

A salary is working income—you stop working, it stops. A percentage of consensus is net worth—it compounds as long as the network lives. The difference is not arithmetic; it is architectural.

## 5 Rich People Manage Their Money, Poor People Mismanage It

“Until you show you can handle what you’ve got, you won’t get any more.”

—T. Harv Eker

### ★ Wealth File #5

*“It’s not about how much money you make—it’s about how much you keep. And how much you keep depends on how well you manage it. Rich people manage their money well. Poor people mismanage their money—or avoid the subject of money altogether.”*

Managing money at the protocol level means managing *precision*. QUG operates with  $10^{24}$  decimal places—24 digits after the decimal point. A single rounding error, multiplied across millions of blocks, can create or destroy millions of coins.

In the world of traditional finance, rounding is an accounting convention. In blockchain, rounding is a consensus-breaking event.



The founder’s money management is not a budget spreadsheet. It is u128 arithmetic with `saturating_mul`:

Listing 4: Precision Money Management (`emission_controller.rs`)

```
1  /// Fixed-point precision multiplier for u128 calculations
2  /// Must satisfy: BASE_ANNUAL_EMISSION * PRECISION < u128::MAX
3  /// 2.625e30 * 1e6 = 2.625e36 < 3.4e38 (margin: ~130x)
4  const PRECISION: u128 = 1_000_000;
5
6  /// Dynamic per-block safety cap: 2x the ideal reward
7  pub fn dynamic_max_reward(era: u64, block_rate_bps: f64) -> u128 {
8      if era >= 64 { return MIN_REWARD; }
9      let target = annual_emission(era);
10     let rate = block_rate_bps.clamp(0.001, 100_000.0);
11     let expected_blocks = (rate * SECONDS_PER_YEAR) as u128;
12     if expected_blocks == 0 { return MIN_REWARD; }
13     let ideal = (target * PRECISION) / expected_blocks / PRECISION;
14     ideal.saturating_mul(2)
15         .clamp(MIN_REWARD, ABSOLUTE_MAX_REWARD_PER_BLOCK)
16 }
```

Every multiplication uses `saturating_mul`—if the result would overflow u128 ( $3.4 \times 10^{38}$ ), it clamps to the maximum instead of wrapping to zero. Every division is checked. Every reward is bounded between `MIN_REWARD` ( $10^{19}$ , about 0.00001 QUG) and `ABSOLUTE_MAX_REWARD_PER_BLOCK` ( $2 \times 10^{24}$ , exactly 2.0 QUG).

In the world of finance, wrapping is theft. Clamping is a circuit breaker.

The coinbase transaction—the transaction that creates new coins—is marked with a signature that cannot be forged:

Listing 5: The Coinbase Marker (`q-types/src/lib.rs`)

```
1  /// Coinbase signature magic marker: [0xC0, 0x1B, 0xA5, 0xE6]
2  /// "COINBASE" encoded in hex + version byte
3  sig_bytes.extend_from_slice(&[0xC0, 0x1B, 0xA5, 0xE6]);
```

`0xC01BA5E6`—“coinbase” spelled in hexadecimal, the programmer’s equivalent of a notary seal. Every node verifies this marker. Every light client can audit it. The money supply is not a database entry that an administrator can edit; it is a cryptographic proof that the entire network validates.

### The Principle

Managing money means managing precision. When your currency has  $10^{24}$  decimal places, a single rounding error is a financial catastrophe. `saturating_mul` is the Rust programmer’s budget discipline.

## 6 Rich People Have Their Money Work Hard for Them

“Rich people have their money work hard for them.  
Poor people work hard for their money.”

—T. Harv Eker

### ★ Wealth File #6

*“The most important thing to understand about money is that it is a tool. Money works for you or you work for money. Rich people see every dollar as a ‘seed’ that can be planted to earn a hundred more dollars, which can then be planted to earn a thousand more.”*

While the founder sleeps, block 87,342 is produced by a miner in Berlin. The emission controller computes the reward: 0.0831 QUG. Of that, 1.9%—0.001579 QUG—flows to `efca1e8c...`. The founder did not wake up. Did not check email. Did not “hustle.”

Three seconds later, block 87,343 is produced by a miner in Seoul. Another 0.001579 QUG.

Every few seconds, somewhere in the world, a CPU cycles through SHA-256 hashes. The gossipsub protocol propagates the block across four bootstrap nodes spanning Europe. Tor circuits encrypt the traffic. Kademia DHT routes the discovery. Dandelion++ obscures the origin.

And 1.9% of every reward, from every block, flows to one address.

This is the ultimate expression of Eker’s principle: money working for you. Not a rental property that needs maintenance. Not a stock that needs monitoring. Not a business that needs employees. *Consensus itself*—the mathematical heartbeat of a decentralized network—doing the work.

A rental property has tenants who call at midnight. Consensus has `saturating_mul`, and it never calls.

The node-operator split (0.1% of the dev fee) incentivizes decentralization: anyone running a node earns a small share, encouraging more nodes, which strengthens the network, which produces more blocks, which generates more fees. A virtuous cycle encoded in seven lines of Rust:

Listing 6: The Virtuous Cycle (`block_producer.rs`)

```
1 // v8.6.1: Split dev fee between founder and node operator
2 let operator_promille = self.node_operator_fee_promille
3   .load(std::sync::atomic::Ordering::Relaxed) as u128;
4 let operator_fee = if operator_promille > 0 {
5     dev_fee_amount.saturating_mul(operator_promille) / 1000
6 } else { 0 };
7 let founder_fee = dev_fee_amount.saturating_sub(operator_fee);
```

### The Principle

The ultimate passive income is not dividends or rent—it is a percentage of consensus. It works while you sleep because it *is* the network sleeping and waking and producing blocks.

## Part III: The Network

---

### 7 Rich People Associate with Positive, Successful People

“Rich people associate with positive, successful people.  
Poor people associate with negative or unsuccessful people.”

—T. Harv Eker

#### ★ Wealth File #7

*“In the real world, your network is your net worth. Energy is contagious. Either you affect people or you infect people. The same holds true for the reverse—either people affect you positively or infect you negatively.”*

A blockchain is, at its core, a network—and Eker’s principle applies literally. Every participant who joins the Q-NarwhalKnight network makes it stronger: more hash power, more geographic distribution, more resilience against attacks. Unlike a social circle, however, the protocol enforces quality mathematically.

By February 2026, the network had grown to:

- **27+ active miners** submitting solutions across multiple continents
- **4 bootstrap nodes**—Beta (Netherlands), Gamma (Germany), Delta (Netherlands), Alpha (testing)—providing network anchor points
- **libp2p gossipsub** for block propagation with cryptographic message signing
- **Kademlia DHT** for peer discovery without centralized registries
- **Dandelion++** gossip for privacy-preserving transaction broadcast

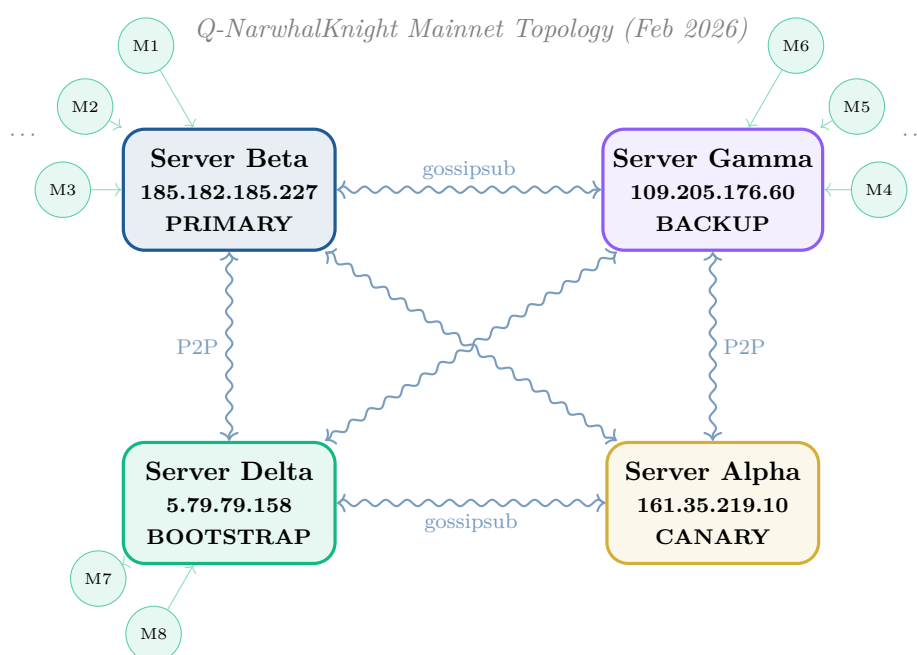


Figure 2: Network topology: 4 bootstrap nodes with 27+ miners connected via gossipsub.

The beauty of a decentralized network as “positive association” is that it is *structural*. You cannot have a lazy node—a node either validates blocks correctly or gets rejected by consensus. There are no negative participants; the protocol filters them out mathematically. Eker advises you to choose your friends wisely; consensus does it for you.

### The Principle

In a blockchain, your network literally is your net worth. Every node that joins makes every other node’s coins more valuable. Consensus is positive association enforced by mathematics.

## 8 Rich People Are Willing to Promote Themselves

“If you believe your product or service can truly fulfill people, it’s your duty to let them know about it.”

—T. Harv Eker

### ★ Wealth File #8

*“Resenting promotion is one of the greatest obstacles to success. People who have issues with selling and promotion are usually broke. Rich people are willing to promote themselves and their value.”*

Most blockchain founders hide their fee structures in footnotes—or worse, in token vesting schedules that only a securities lawyer can parse. The Q-NarwhalKnight founder did the opposite: 71 academic whitepapers. A public website at [quillon.xyz](https://quillon.xyz). Open-source code. A transparent 1.9% fee that anyone can audit by reading `block_producer.rs` line 1139.

This is not arrogance. It is what Eker calls “promoting your value.” The 1.9% fee is not hidden because hiding it would be a *poverty* mindset—the belief that earning money from your creation

is somehow shameful.

The promotion includes:

- **71 academic whitepapers** covering consensus, privacy, quantum cryptography, economics, DEX design, AI integration, and governance
- **quillon.xyz**—a full-featured web wallet with real-time SSE streaming, transaction history, and network statistics
- **A download page** where anyone can get the node binary with a single `wget` command
- **Complete transparency:** every constant, every fee split, every wallet address is visible in the source code

The 72nd paper is the one you are reading now.

### The Principle

Hiding your fee is a poverty mindset. Publishing 72 papers explaining your architecture, your economics, and your fee structure is the blockchain equivalent of Eker's self-promotion—shameless, transparent, and necessary. If you built something worth using, silence is not humility; it is sabotage.

## Part IV: The Launch

### 9 Rich People Act in Spite of Fear

“Action is the bridge between the inner world and the outer world.”

—T. Harv Eker

#### ★ Wealth File #9

*“Comfort is the enemy of achievement. A truly rich person is willing to act in spite of fear. The key to success is not to try to eliminate fear—it is to act in spite of it.”*

February 22, 2026. 12:00 UTC. Genesis timestamp 1771761600.

The launch procedure reads like a controlled demolition—because it is one:

1. **T–30 min:** Stop *all* old servers simultaneously. Both must be down before either new one starts—this prevents HTTP state-sync contamination from old to new.
2. **T–20 min:** Update service files with new network ID, new database path, new encryption passphrases.
3. **T–15 min:** Generate fresh encryption keys from `/dev/urandom`.
4. **T–10 min:** Start Beta first (the bootstrap node). A new libp2p identity auto-generates.
5. **T–5 min:** Start Gamma. Capture its new peer ID.
6. **T±0:** Genesis timestamp reached. Mining begins automatically. The first block is produced.

The fear is not irrational. It is specific and technical:

- What if a bug in `balance_consensus.rs` creates coins from nothing?
- What if the emission controller’s error correction oscillates and mints  $34\times$  the intended supply—*again*?
- What if a height regression wipes the chain at block 5,000?
- What if RocksDB’s block cache eats all RAM and OOM-kills the bootstrap node during the first hour?

Every one of these has happened before. Every one was fixed. But the fix is in *this* binary, compiled *this* morning, and if there is a regression that the 4,000 tests did not catch—

You act in spite of fear. You type `systemctl start q-api-server` and watch the first block appear in the logs:

```
1 [2026-02-22T12:00:03Z] Block #1 produced: reward=0.0831 QUG
2 [2026-02-22T12:00:03Z] Coinbase: 98.0% -> miner, 1.9% -> founder
3 [2026-02-22T12:00:03Z] Network: mainnet2026.2 | Genesis: 1771761600
4 [2026-02-22T12:00:03Z] P2P: 2 peers connected via gossipsub
```

### The Principle

Every mainnet launch is an act of faith disguised as engineering. You cannot eliminate the fear of a consensus bug that destroys everything. You can only act in spite of it—after writing 4,000 tests.

## 10 Rich People Constantly Learn and Grow

“Every master was once a disaster.”

—T. Harv Eker

### ★ Wealth File #10

*“You can be right or you can be rich, but you can’t be both. Rich people constantly learn and grow. Poor people think they already know.”*

The version history tells the story of continuous growth:

Table 2: The Journey from Prototype to Production

| Version     | Date      | Lesson                                                               |
|-------------|-----------|----------------------------------------------------------------------|
| v0.2.0-beta | 2025      | First compilation. Nothing works.                                    |
| v1.4.1-beta | Late 2025 | Mainnet safety: height-gated validation rules                        |
| v3.3.7-beta | Jan 2026  | Connection warmup for new nodes                                      |
| v7.1.5      | Feb 2026  | Configurable dev fee in basis points                                 |
| v7.2.8      | Feb 2026  | Dynamic max reward—fixed 92.5% under-emission                        |
| v7.4.1      | Feb 2026  | Height regression fix: <code>put_sync</code> with <code>fsync</code> |
| v8.0.0      | Feb 2026  | 23-crate comprehensive update                                        |
| v8.5.9      | Feb 2026  | Miner version fix, thread retry, QCredit vault                       |
| v8.6.4      | Feb 2026  | Current production—emission stable                                   |

Each version is a lesson Eker would recognize: the version where floating-point arithmetic was replaced with integer-only computation (“you can be right or you can be rich”). The version where the correction-factor smoothing was raised from 0.5 to 0.8 because wall-clock rate measurement made it safe to be more aggressive (“constant learning”). The version where `cargo clean -package q-api-server` became mandatory after changing constants, because incremental compilation caches old values (“every master was once a disaster”).

In traditional software, version numbers are marketing. In blockchain, they are scar tissue—and scar tissue is stronger than the original skin.

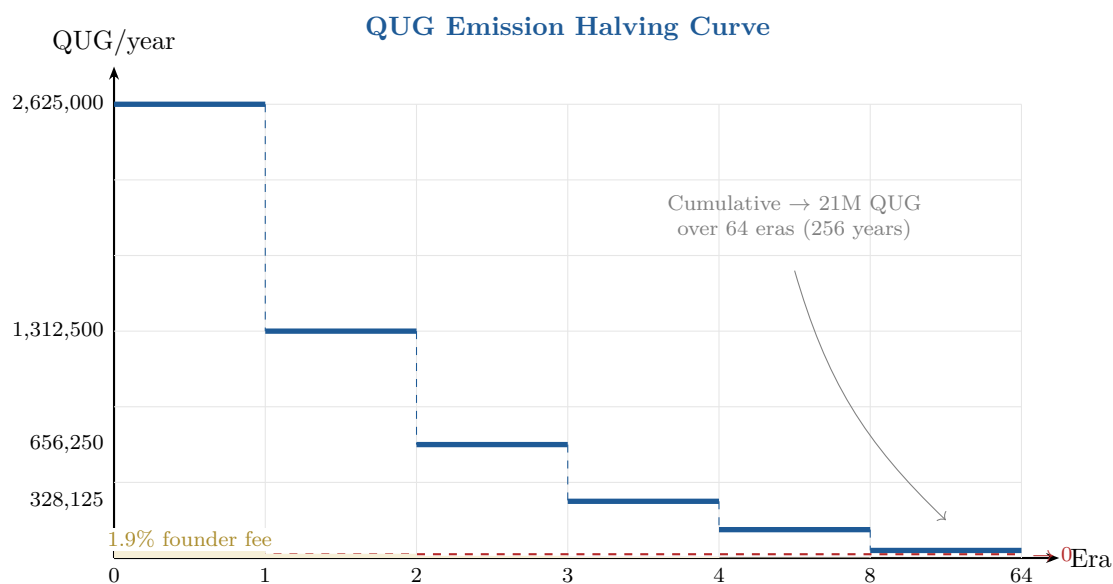


Figure 3: Emission schedule: annual emission halves every 4 years for 64 eras, asymptotically approaching the 21M QUG supply cap. The gold band represents the 1.9% founder fee.

The 4,000-test suite is perhaps the purest expression of “constant learning”: every bug that was found became a test that ensures it can never happen again. The tests are not bureaucracy—they are *institutional memory*, a system that learns from its own mistakes and compiles the lessons into guarantees.

### The Principle

Version numbers are a growth journal. Each increment represents a bug found, a lesson learned, a system made more resilient. The founder who stops learning stops growing—and in blockchain, that means the network dies.



## Part V: The Long View

### 11 After the Last Coin: The Post-Emission Economy

“The goal of creating wealth is not to just have money.  
The goal is freedom.”

—T. Harv Eker

#### ★ Wealth File #11

*“Rich people think long term. They balance their spending on enjoyment today with investing for freedom tomorrow. Poor people think short term. They run their lives based on immediate gratification.”*

Every emission schedule has an endpoint. Bitcoin’s last satoshi will be mined around the year 2140. QUG’s last fractional coin will be emitted around the year 2282. What happens to the 1.9% founder fee when there are no more block rewards to take 1.9% of?

The answer is built into the design: transaction fees.

As emission decreases across 64 halving eras, the network transitions from an emission-funded model to a fee-funded model—the same transition Bitcoin faces, but with 256 years of runway instead of 120. During this transition:

- **Early eras (0–8):** Block rewards dominate. The 1.9% fee provides steady founder income from emission.
- **Middle eras (8–32):** Emission shrinks. If the network has achieved adoption, transaction fees grow to compensate. The DEX, smart contracts, and stablecoin system all generate on-chain activity—and fees.
- **Late eras (32–64):** Emission is negligible. Transaction fees sustain miners and, through them, the founder fee. The network is self-sustaining or it is not—there is no central bank to print more.

This is the honest answer: the 1.9% fee’s long-term value depends entirely on whether the network achieves real adoption. It is not a guaranteed income stream; it is an *incentive-aligned bet* that the founder’s continued work will make the network valuable enough to generate transaction volume.

Eker would approve: the founder’s wealth is tied directly to the network’s success. There is no golden parachute. No vesting cliff after which the founder can walk away. The incentive alignment is permanent and immutable—encoded in consensus, not in a contract that a lawyer can renegotiate.

#### The Principle

A truly long-term financial blueprint accounts for its own expiration. The 1.9% fee is not forever—it lasts exactly as long as emission does. After that, value must come from utility, not from minting. That constraint is a feature, not a bug.

## 12 Governance: Who Controls the Thermostat?

“If you want to change the fruits, you will first have to change the roots.”

—T. Harv Eker

A natural question arises: if the 1.9% fee is hardcoded into consensus, who can change it? What if the community decides the fee should be 1.0%, or 0%, or 3%?

The current answer is direct: the fee is a constant in `block_producer.rs`. Changing it requires a code change, a new binary, and every node operator choosing to upgrade. This is governance by rough consensus—the same model Bitcoin uses for its block size, its halving schedule, and its scripting language.

No DAO vote can override it. No token-weighted governance can force it. The fee changes only if the people running the nodes choose to run a binary that changes it. This is slower than on-chain governance, more conservative than a multisig—and, the founder would argue, more resistant to capture.

The `dev_fee_bps` variable is an `AtomicU64`, which means it *can* be adjusted at runtime via configuration. But the default is 190 basis points, and changing it requires deliberate, transparent action by the node operator. No silent extraction. No backdoor.

### The Principle

The best governance is the kind where changing the rules requires convincing every participant to update their software. It is slow, it is conservative, and it is the most censorship-resistant form of governance ever invented.

## Epilogue

### *The Quantum Millionaire's Creed*



T. Harv Eker ends *Secrets of the Millionaire Mind* with declarations—statements you speak aloud to reprogram your financial blueprint. Here are ten declarations for the blockchain builder.

### The Quantum Millionaire's Ten Declarations

1. **“My code is my financial blueprint.”**  
I do not inherit a money story from society. I write my own—in Rust, with `u128` precision, compiled to machine code, verified by 4,000 tests.
2. **“I think big: 80 crates, 256 years, 21 million coins.”**  
I do not fork someone else's vision. I build from first principles, even when the world says it is too ambitious, too complex, too early for quantum threats that do not yet exist.
3. **“I am committed, not merely interested.”**  
I fix the  $34\times$  emission bug at 3 AM. I diagnose the 26.9 GB memory leak. I count zeros in 31-digit constants. Interest takes weekends off. Commitment does not.
4. **“I focus on net worth, not working income.”**  
I earn a percentage of every block reward—not a salary that stops when I stop. Consensus works while I sleep. My income is a function of the network's hash rate, not my hours.
5. **“I manage my money with saturating\_mul.”**  
I use  $10^{24}$  decimal precision. I clamp every calculation between `MIN_REWARD` and `ABSOLUTE_MAX_REWARD`. I mark every coinbase with `0xC01BA5E6`. Overflow is not a rounding error—it is a financial catastrophe I refuse to allow.
6. **“I have my money work hard for me.”**  
Every miner on every continent sends a fraction to `efca1e8c...` with every block. I did not ask them to. Consensus asked them to. The code asked them to. The math asked them to.
7. **“My network is my net worth.”**  
27 miners. 4 bootstrap nodes. Gossipsub, Kademlia, Dandelion++. Every node that joins makes every coin more valuable. I do not compete with my network—I *am* my network.
8. **“I promote my value transparently.”**  
72 whitepapers. Open source. A public fee structure anyone can audit at `block_producer.rs` line 1139. I do not hide my fee because hiding income is a poverty mindset.
9. **“I act in spite of fear.”**  
I type `systemctl start q-api-server` on mainnet launch day knowing that a single bug in `balance_consensus.rs` could destroy everything. I act anyway—after writing 4,000 tests.
10. **“I constantly learn and grow.”**  
v0.2.0-beta to v8.6.4. Hundreds of versions. Each bug a lesson, each lesson a test, each test a guarantee that the mistake can never recur. I do not fear mistakes—I compile them into wisdom.

---

*“The blockchain does not care about your feelings.”*

*It cares about your constants.*

*And your constants had better be right—  
because 27 miners are counting on them.”*

—The Q-NarwhalKnight Founder



**Q-NarwhalKnight** — Quantum-Enhanced DAG-BFT Consensus  
<https://quillon.xyz>

*Inspired by T. Harv Eker’s “Secrets of the Millionaire Mind”  
and the real journey of building a blockchain from zero to mainnet.*

February 2026 · Version 1.1 · Paper #72